

Assignment 3

Name(s): Licheng Xu, Libin Wang

NetID(s): lx8, libin2

Team name on Kaggle leaderboard: Libin Wang & Licheng Xu

For each of the sections below, your reported test accuracy should approximately match the accuracy reported on Kaggle. It is fine to have a small deviation. Report both accuracies if they are different.

Self-supervised Learning on CIFAR10

We record test accuracy in last epoch and test accuracy with lowest loss in the following parts. For the two accuracies, their values differ by around 0.5%, and the corresponding difference in the number of training rounds is around 5, so there are only small fluctuations. We choose to save the model in last epoch for stability.

1) Rotation training

Num_epochs	45
Decay_epochs	20
Init_lr	0.001
Test accuracy in last epoch	81.31%
Test accuracy with lowest loss [Epoch 44]	81.18%
Test accuracy (Kaggle)	80.68%

For the self-supervised rotation task, our optimizer originally chose SGD, but found that the test accuracy was less than 78% with the parameters provided. Therefore, we switch to Adam optimizer and achieve a test accuracy higher than the baseline with the parameters provided

By observing the change of the test accuracy with the number of training epochs, we consider the initial learning rate is set too high, and the model does not converge completely at the end of the training. So we change the initial learning rate from 0.01 to 0.001, and the final test accuracy has an improvement of about 2%.

2) Fine-tuning late layers

Partially fine-tuning on the pre-trained model:

Num_epochs	35
Decay_epochs	15
Init_lr	0.005
Test accuracy in last epoch	69.98%
Test accuray with lowest loss [Epoch 29]	70.12%
Test accuracy (Kaggle)	69.98%

Partially supervised training from scratch:

Num_epochs	50
Decay_epochs	20
Init_lr	0.005
Test accuracy in last epoch	49.33%
Test accuray with lowest loss [Epoch 44]	49.49%

In Part2, we choose the same optimizer and loss function as in Part1, and used the same hyperparameters at the beginning. We found that the pre-trained based model had a much higher test accuracy than the randomly initialized based model. We then increased the number of training epochs for the randomly initialized based model, and found that the test accuracy did not improve significantly after the 37th round, thus excluding the possibility that the model was insufficiently trained to some extent. Therefore, the pre-trained based model converges faster and the accuracy of the test is higher when only the topping layers are updated.

However, it is worth noting that when updating the parameters of the model, we use the parameters of layer4 in the pre-trained model as the warm start. This is slightly different from the Gidaris et al. paper, where only the parameters of the backbone part (i.e. layers1-3) of the pre-trained model are retained; for the new topping layers, even though the architecture is the same as the pre-trained model. The paper argues that the topping layers are “more and more specific on the self-supervised task of rotation prediction”, and thus their bias towards the self-supervised task can be avoided to some extent by direct random initialization in downstream training.

In this case, we try randomly initializing the parameters of layer4 and train with the same hyperparameters. We find that the test accuracy did not differ much from the original, thus proving in some way that the original parameters of the topping layers have little effect on the new classification task. Below is our result.

Num_epochs	35
Decay_epochs	15
Init_lr	0.005
Test accuracy	70.09%

3) Fully supervised learning

Fully fine-tuning on the pre-trained model:

Num_epochs	35
Decay_epochs	15
Init_lr	0.005
Test accuracy in last epoch	86.00%
Test accuracy with lowest loss [Epoch 32]	85.82%
Test accuracy (Kaggle)	86.00%

Fully supervised training from scratch:

Num_epochs	35
Decay_epochs	15
Init_lr	0.005
Test accuracy in last epoch	84.35%
Test accuracy with lowest loss [Epoch 33]	84.27%

As in Part1 and 2, we use the same optimizer and loss function in Part3, and apply the same hyperparameters to compare the test accuracies. We find that when updating the whole model parameters, the difference in accuracy between pre-trained based model and randomly initialized based model is very small, only about 1.5%;

Also, comparing the performance of the pre-trained based model under two parameter tuning strategies, we can find that the model performs much better on the test when tuning all model parameters. We believe this may be due to the simplicity of the rotation task (4-class classification) compared to the CIFAR10 classification task (10-class classification). If the parameters of backbone part are not tuned, it may not be enough to express the target task.

4) With a more advanced architecture

4.1) New rotation learning

Num_epochs	60
Decay_epochs	20
Init_lr	0.001
Test accuracy in last epoch	86.19%
Test accuracy with lowest loss [Epoch 56]	86.70%
Test accuracy (Kaggle)	86.68%

For more advanced architecture, we choose Resnet50 from pytorch. At the same time, we use AdamW as a new optimizer to improve the generalization ability of the model by introducing weight decay.

Initially, we trained the model with initial learning rate equal to 0.01, and increased the number of training epochs given that the model has a larger number of parameters than the original one. However, we found that the learning rate of the model was too large: the test accuracy kept “oscillating” at the end of the training period. Therefore, we reduced the learning rate, but found that the accuracy of the model had been stabilized at 70% or so, and did not even reach the baseline in Part1;

Through online search, we learn that the Resnet model is designed for the ImageNet dataset, in which the resolution of the images is much larger than that of the CIFAR10 dataset. Therefore, we think the reason for the poor performance of the model may be: the initial stride and kernel size of the conv1 layer are too large, which leads to the image keeping too few features in the later layers. Thus, we adjust the stride of the conv1 layer from 2 to 1, the kernel size from 7 to 3. Eventually we achieve an accuracy of more than 86%.

4.2) New fully fine-tuning on the pre-trained model

Num_epochs	20
Decay_epochs	10
Init_lr	0.001
Test accuracy in last epoch	90.66%
Test accuracy with lowest loss [Epoch 17]	90.80%
Test accuracy (Kaggle)	90.66%

The accuracy comparison in Parts 2-3 shows that the best training setting is: “Train a network on Rotation, then use these weights as a warm start. Finetune the full network.”

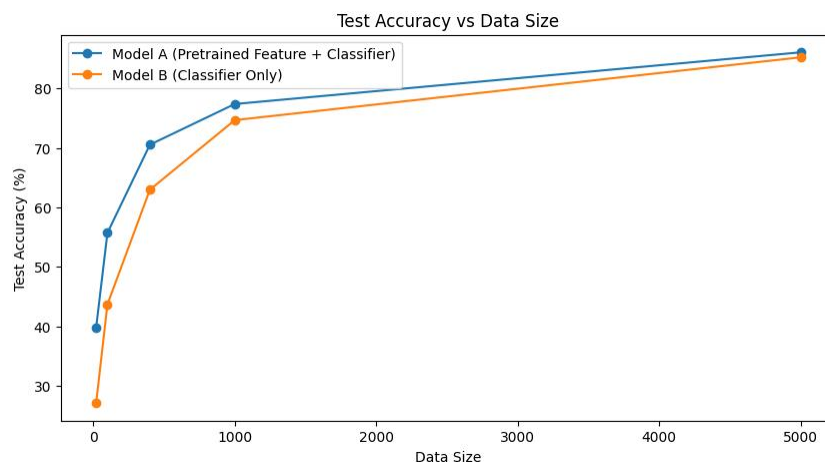
For the optimizer, we use the same AdamW as in the new pre-training model. And for the loss function, we add label smoothing to CrossEntropyLoss in the hope of further improving the model's generalization ability.

The learning rate of the new model is the same as in the pre-training, and the same number of training epochs as in Part3 are used at the beginning. However, we observed the changes in the test accuracy and found that the model was close to the peak accuracy after a dozen rounds of training. In the following training, the accuracy did not change much, but the average test loss tended to increase. We think this may be due to the following facts. First, the architecture of the new model is larger than the original parameter count, and more general features of the image are learned in the pre-training phase. Second, the use of label smoothing further accelerates the convergence speed of the model. Based on this, we reduce the number of training rounds to 20, and achieve more than 90% accuracy.

5) Extra credit

5.1) Drawing plot

The code is in the `extra_credit_part1.ipynb`



We revised the dataloader to only use a subset of the training set. Our code promises that the data size for each label is equal. For Model A, we load the rotated model as a pre-trained feature and then train the net. We use the same data sizes as in the essay, which are [20, 100, 400, 1000, 5000]. We can see in the plot for data sizes 20 and 100 that there is a significant difference between Model A and Model B, but as the data sizes grow larger, the difference shrinks gradually. When the data size is 5000, there is approximately no difference regarding whether we pretrain the model with the rotate mission. The result is similar to the plot in the essay experiment.

5.2) Larger Dataset (ImageNette)

The code is in the `extra_credit_part2.ipynb`

Num_epochs	45
Decay_epochs	15
Init_lr	0.001
Test accuracy in last epoch	59.64%
Test accuracy with lowest loss [Epoch 39]	59.80%

We try to perform the rotation task on the larger ImageNette dataset. The dataset has a much smaller data volume compared to CIFAR10, but the images themselves have a much larger resolution. We resize the images to 224×224 , the same size as the input during ImageNet training.

In order to improve the generalization ability of the model, we choose Resnet18 as initial model and add a Dropout layer in front of the fc layer. We also use AdamW as optimizer and CrossEntropyLoss as loss function. Due to the computational power limitations of the computer, the final number of epochs is set to 45 and test accuracy achieves nearly 60%.